

Ein Programm schreiben – Teil 1

Abstrakter Entwurf bis Hauptcode

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 24. November 2025 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan  überlebt den ersten Kontakt  mit der Realität .

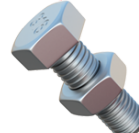
Frei nach Helmuth Karl
Bernhard von Moltke

Ein Programm schreiben

 Plan für diese Woche

 Abstrakter Entwurf

 Eine Klasse entwickeln

 Testcode

 Hauptcode

 Entwicklungszyklen

 Pair Programming

 Schleifen

 Elementare Typen casten

 Spaß

📖 Ein Programm schreiben

🔗 Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

💡 Welche Art von Diagramm eignet sich für einen abstrakten Entwurf?

👤 Welche Schritte sollten in welcher Reihenfolge durchgeführt werden, um ein Programm zu schreiben?

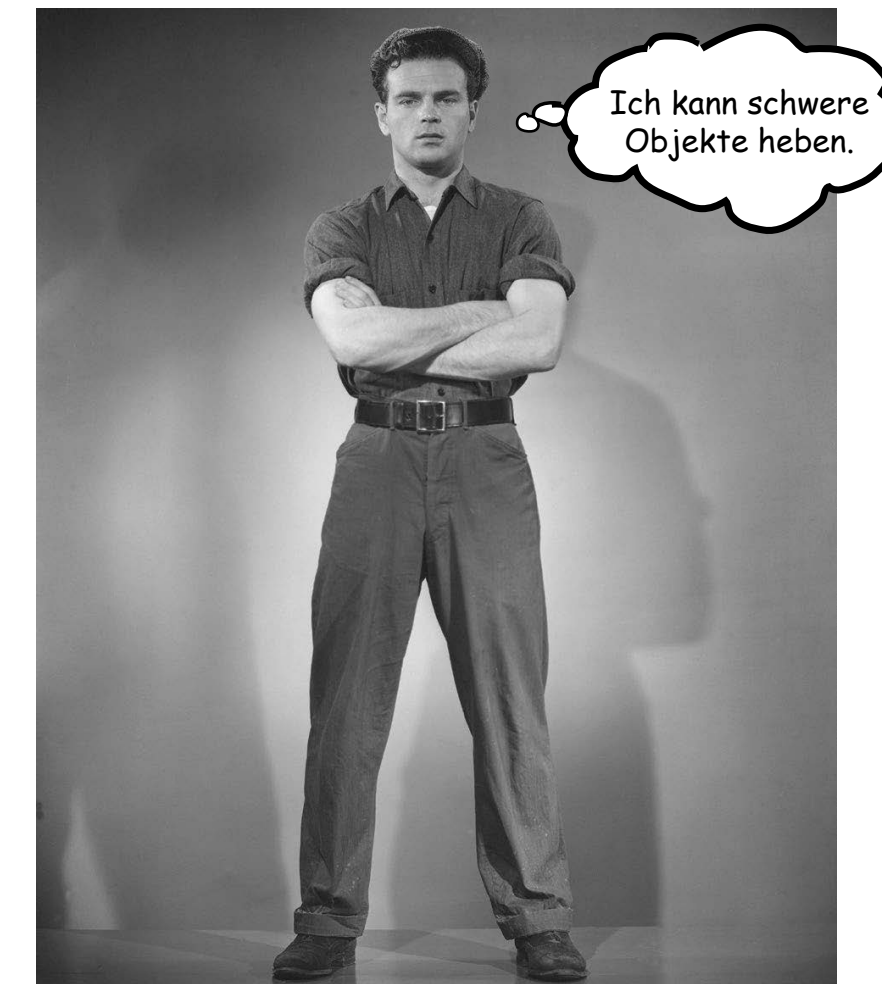
🔧 Was ist Testcode und wann sollte er geschrieben werden?

🔧 Was ist im Hauptcode eines Programms enthalten?

👤 Welche Fragen haben Sie zu diesem Kapitel?

5 Ein Programm schreiben

Methoden mit Superkräften



Jetzt wollen wir unsere Methoden mal etwas aufmöbeln. Wir haben uns mit Variablen beschäftigt, mit ein paar Objekten herumgespielt und ein bisschen Code geschrieben. Aber wir waren schwach. Wir brauchen Werkzeuge. Wie **Operatoren**. Und wir brauchen mehr Operatoren, um interessante Dinge zu tun, zum Beispiel *bell**en*. Und **Schleifen**. Wir brauchen Schleifen – mehr als diese mickrigen *while*-Schleifen. Eigentlich brauchen wir **for**-Schleifen. Sie könnten nützlich sein, um **Zufallszahlen zu erzeugen**. Am besten lernen wir das gleich mit. Und warum lernen wir das alles nicht, indem wir sofort etwas Reales *bauen*? Dann sehen wir, wie es ist, ein Programm von Grund auf neu zu coden (und zu testen). **Vielleicht ein Spiel** wie Schiffe versenken. So eine schwere Aufgabe müssen wir erst einmal gestemmt bekommen. Deshalb lassen wir uns auch *zwei* Kapitel Zeit, um die Aufgabe abzuschließen. In diesem Kapitel schreiben wir eine einfache Version. In Kapitel 6 bauen wir dann die De-luxe-Fassung – *mithilfe der Java-Bibliothek*.



Abstrakter Entwurf



Ratespiel

Abstrakter Entwurf

 **Ziel:** Eine zufällige Zahl erraten.

 **Vorbereitung:** Das Programm erzeugt zum Spielstart eine Zufallszahl. Diese soll erraten werden.

Spielweise:

 Das Programm fragt nach einer Obergrenze für die Zufallszahl.

 Das Programm lässt dann so lange Rateversuche zu, bis die Zahl erraten wurde.

 Wurde die Zahl erraten, gibt das Programm die Anzahl der Rateversuche aus.





Flussdiagramm



Abstrakter Entwurf



Ziel: Eine zufällige Zahl erraten.



Vorbereitung: Das Programm erzeugt zum Spielstart eine Zufallszahl. Diese soll erraten werden.



Spielweise:



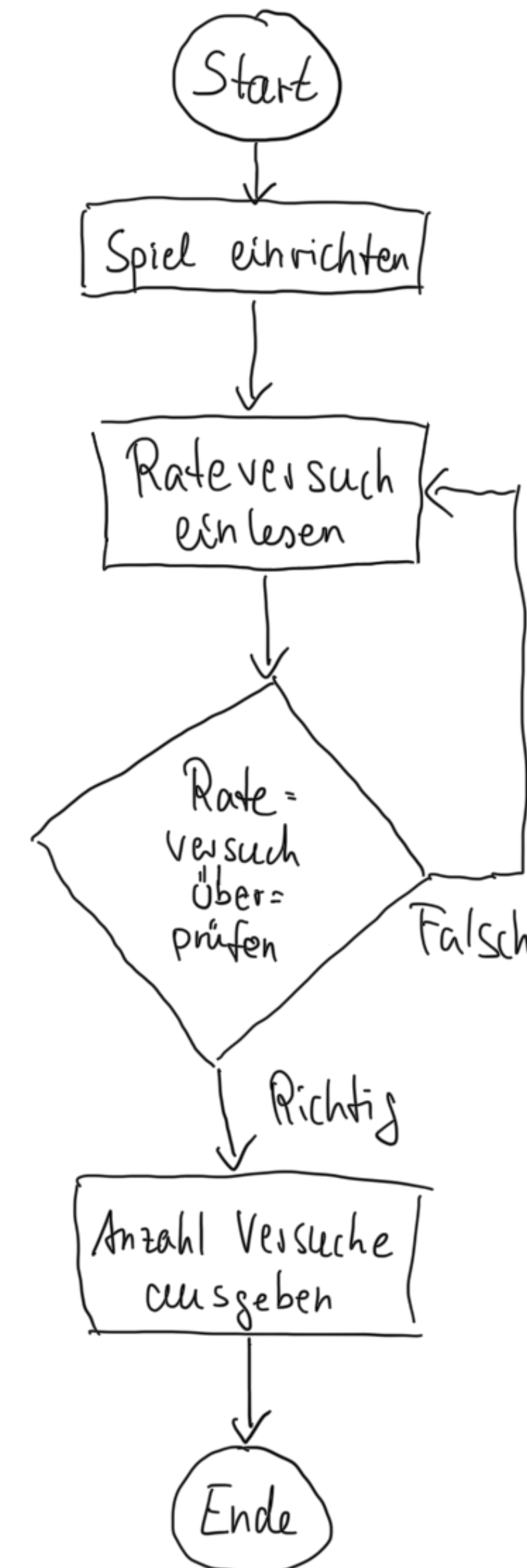
Das Programm fragt nach einer Obergrenze für die Zufallszahl.



Das Programm lässt dann so lange Rateversuche zu, bis die Zahl erraten wurde.



Wurde die Zahl erraten, gibt das Programm die Anzahl der Rateversuche aus.



Kreis steht für
Anfang / Ende

Rechteck steht
für Aktion

Raute kennzeichnet
Verzweigung /
Entscheidung



Grundidee



Abstrakter Entwurf

1. Benutzer:in startet das Spiel

- A. Spiel fragt nach einer Obergrenze für die Zufallszahl
- B. Spiel erzeugt eine Zufallszahl zwischen 1 und Obergrenze

2. Spiel läuft

Solange die Zahl nicht erraten wurde, folgende Schritte wiederholen:

- A. Benutzer:in zu einem Rateversuch auffordern (eine Zahl)
- B. Rateversuch mit Zufallszahl vergleichen und Ergebnis ausgeben

3. Das Spiel endet

Anzahl der Rateversuche ausgeben.



Kreis steht für
Anfang / Ende

Rechteck steht
für Aktion

Raute kennzeichnet
Verzweigung /
Entscheidung

🧙 „Genau, auch super
hilfreich für Codeopolis.“

„Ha! Diese Vorgehensweise hilft mir, **wie**
eine Programmierer:in zu denken!“

Merle – aufmerksame Student:in im 1. Semester



! ? Quiz

💡 **Abstrakter Entwurf**

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Sie beginnen, Ihr Java-Programm zu implementieren, indem Sie den Java-Code schreiben.

Ausgangspunkt für Ihr Java-Programm sollte ein abstrakter Entwurf sein.

Ein abstrakter Entwurf kann zeigen, welche Fähigkeiten ein Programm beherrschen muss.

Richtig!

Ein abstrakter Entwurf hilft zu bestimmen,
welche Arten von Objekten für ein
Programm benötigt werden.

Richtig!

Ein Flussdiagramm visualisiert komplizierte Informationen und stellt die Schritte eines Programms oder Algorithmus mit Hilfe verschiedener geometrischer Formen und Pfeile dar.

Richtig!

Ein Rechteck stellt den Anfang oder ein
Ende dar.

Das wäre ein Kreis. Ein Rechteck stellt eine Aktion dar.

Eine Raute kennzeichnet eine Verzweigung
oder eine Entscheidung.

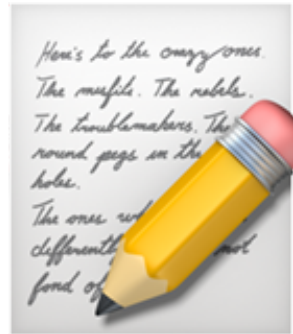
Richtig!



Welcher **eine** Gedanke zu



„Abstrakter Entwurf“ war
für Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und



Sortieren



Eine Klasse entwickeln

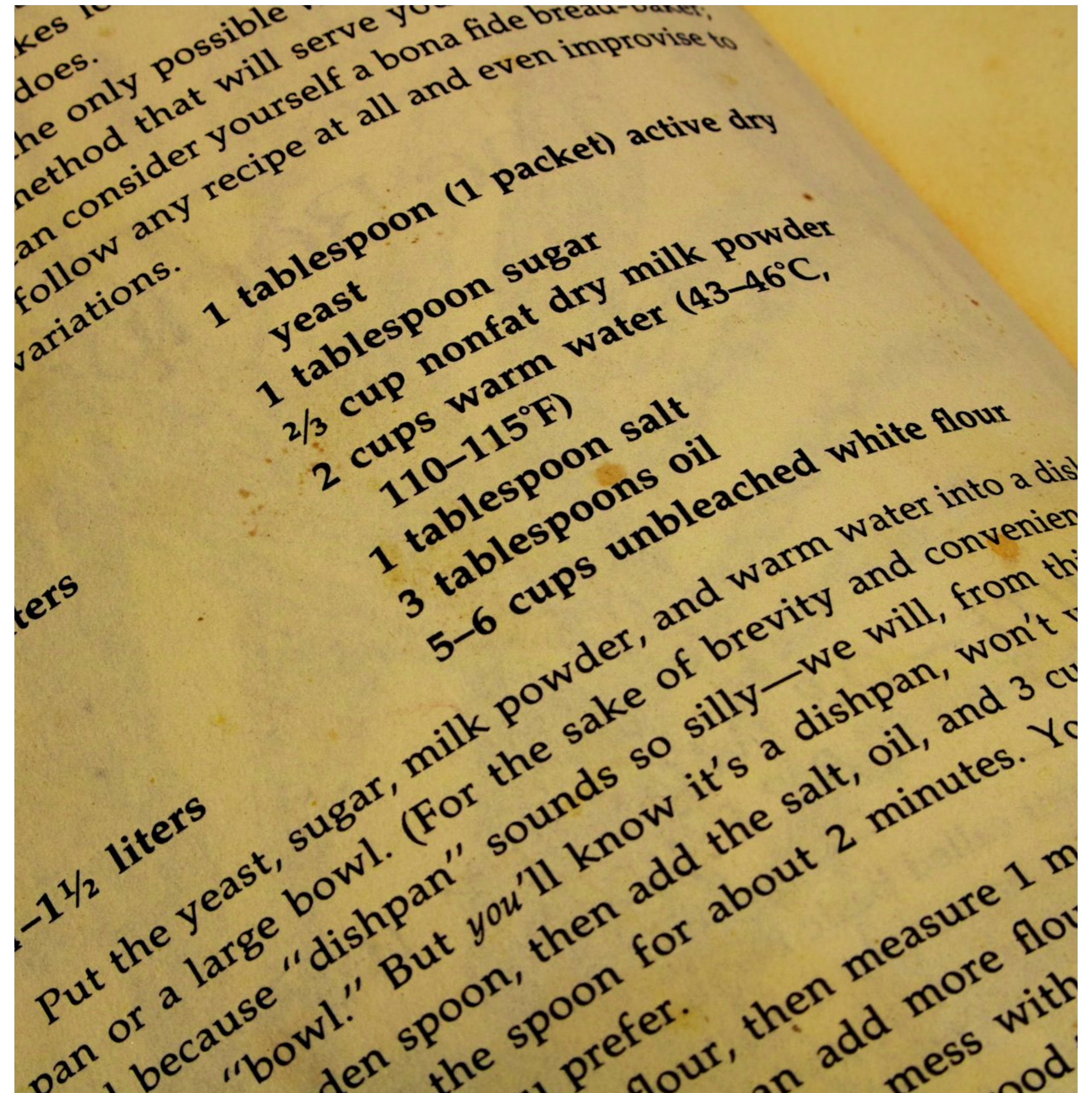


Kochrezept



Eine Klasse entwickeln

☑ Welche Klassen gibt es?



»Um welche **Dinge** geht es in diesem Programm, wer sind die **Hauptakteure**?« Erst dachte er an die **Formen**, dann fielen ihm aber noch andere Dinge ein: die **Benutzer**, der **Klang**, das **Click-Event**.

Brad – Objektorientierung-Entwickler





Kochrezept

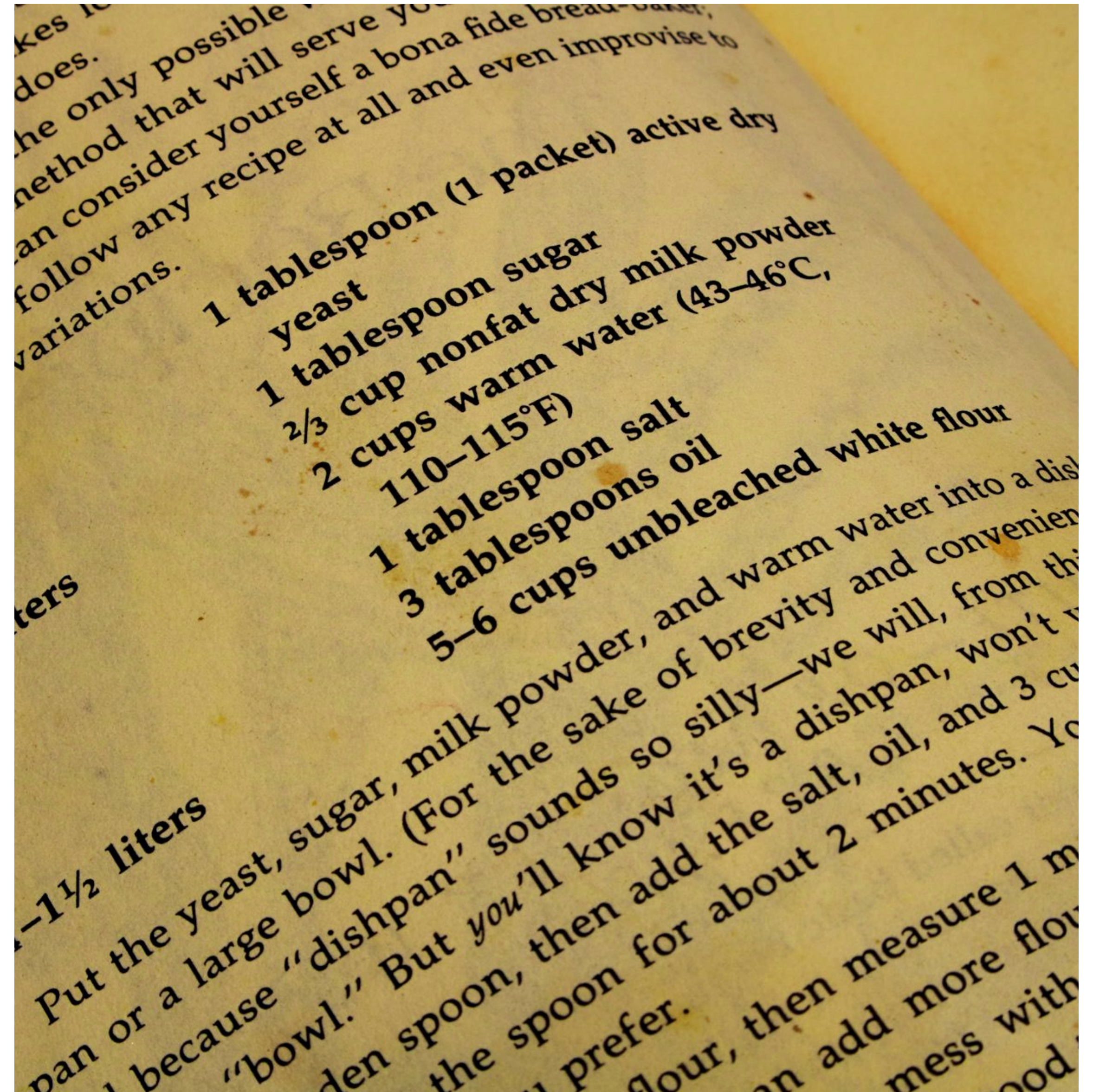


Eine Klasse entwickeln

✓ Welche Klassen gibt es?

🔄 Dann für jede Klasse:

- ✓ Was soll die Klasse tun?
- ✓ Welche Attribute und Methoden hat die Klasse?
- ✓ **Vorcode** für die Klasse schreiben.
- ✓ **Testcode** für die Methoden schreiben.
- ✓ Die Klasse mit „richtigem“ **Hauptcode** implementieren.
- ✓ Verhalten/Methoden der Klasse testen.
- ✓ Die Klasse debuggen und ggf. den Code überarbeiten.



3 Arten von Code

👤 Eine Klasse entwickeln

1. 🖋️ **Vorcode:** Sogenannter Pseudocode, der Ihnen hilft, sich auf die Logik des Programms statt auf die Java-Syntax zu konzentrieren.

💡 Also auf das „**Was**“ statt auf das „**Wie**“.

2. 🛠️ **Testcode:** Klasse oder Methoden, die sicherstellt, dass der Hauptcode das Richtige tut.

3. 🛠️ **Hauptcode:** Tatsächliche Implementierung der Klasse mit passendem Java-Code.

💡 Hier kommt das „**Wie**“.



„Und wie sieht so ein Vorcode aus?“

Noel – neugierige Student:in im 1. Semester





Grundidee des groben Spielablaufs



Eine Klasse entwickeln

1. Benutzer:in startet das Spiel

- A. Spiel fragt nach einer Obergrenze für die Zufallszahl
- B. Spiel erzeugt eine Zufallszahl zwischen 1 und Obergrenze

2. Spiel läuft

Solange die Zahl nicht erraten wurde, folgende Schritte wiederholen:

- A. Benutzer:in zu einem Rateversuch auffordern (eine Zahl)
- B. Rateversuch mit Zufallszahl vergleichen und Ergebnis ausgeben

3. Das Spiel endet

Anzahl der Rateversuche ausgeben.



Grundidee → Pseudocode 1/3



Eine Klasse entwickeln

1. Benutzer:in startet das Spiel

- A. Spiel fragt nach einer Obergrenze für die Zufallszahl
- B. Spiel erzeugt eine Zufallszahl zwischen 1 und Obergrenze

2. Spiel läuft

Solange die Zahl nicht erraten wurde, folgende Schritte wiederholen:

- A. Benutzer:in zu einem Rateversuch auffordern (eine Zahl)
- B. Rateversuch mit Zufallszahl vergleichen und Ergebnis ausgeben

3. Das Spiel endet

Anzahl der Rateversuche ausgeben.

1. Benutzer:in startet das Spiel

- *Spiel fragt nach einer Obergrenze für die Zufallszahl*
 - Gib aus: "Bitte eine Obergrenze für die Zufallszahl eingeben:"
 - Lies die Eingabe der Obergrenze von der Benutzer:in ein
 - Speichere die Obergrenze in der Variable `obergrenze`
- *Spiel erzeugt eine Zufallszahl zwischen 1 und Obergrenze*
 - Erzeuge eine Zufallszahl zwischen 1 und der angegebenen `obergrenze`
 - Speichere die Zufallszahl in der Variable `zielZahl`
 - Setze die Variable `zähler` für die Rateversuche auf `0`



Grundidee → Pseudocode 2/3



Eine Klasse entwickeln

1. Benutzer:in startet das Spiel

- A. Spiel fragt nach einer Obergrenze für die Zufallszahl
- B. Spiel erzeugt eine Zufallszahl zwischen 1 und Obergrenze

2. Spiel läuft

Solange die Zahl nicht erraten wurde, folgende Schritte wiederholen:

- A. Benutzer:in zu einem Rateversuch auffordern (eine Zahl)
- B. Rateversuch mit Zufallszahl vergleichen und Ergebnis ausgeben

3. Das Spiel endet

Anzahl der Rateversuche ausgeben.

2. Spiel läuft

– Solange die Zahl nicht erraten wurde

- Benutzer:in zu einem Rateversuch auffordern
 - Gib aus: "Rate die Zahl:"
 - Lies den Rateversuch der Benutzer:in ein
 - Speichere den Rateversuch in der Variable `rateversuch`
 - Erhöhe den `zähler` für die Rateversuche um 1
- Rateversuch mit Zufallszahl vergleichen und Ergebnis ausgeben
 - Wenn der `rateversuch` kleiner ist als die `zielZahl`:
 - Gib aus: "Die gesuchte Zahl ist größer."
 - Andernfalls, wenn der `rateversuch` größer ist als die `zielZahl`:
 - Gib aus: "Die gesuchte Zahl ist kleiner."
 - Andernfalls (`rateversuch` ist gleich der `zielZahl`):
 - Gib aus: "Richtig geraten! Die gesuchte Zahl war: " + `zielZahl`
 - Beende die Schleife



Grundidee → Pseudocode 1/3



Eine Klasse entwickeln

1. Benutzer:in startet das Spiel

- A. Spiel fragt nach einer Obergrenze für die Zufallszahl
- B. Spiel erzeugt eine Zufallszahl zwischen 1 und Obergrenze

2. Spiel läuft

Solange die Zahl nicht erraten wurde, folgende Schritte wiederholen:

- A. Benutzer:in zu einem Rateversuch auffordern (eine Zahl)
- B. Rateversuch mit Zufallszahl vergleichen und Ergebnis ausgeben

3. Das Spiel endet

Anzahl der Rateversuche ausgeben.

3. Das Spiel endet

- Gib aus: "Du hast die Zahl in " + `zähler` + " Rateversuchen erraten."





Pseudocode für das Zahlenratespiel



Eine Klasse entwickeln

1. Benutzer:in startet das Spiel

- Spiel fragt nach einer Obergrenze für die Zufallszahl
 - Gib aus: "Bitte eine Obergrenze für die Zufallszahl eingeben:"
 - Lies die Eingabe der Obergrenze von der Benutzer:in ein
 - Speichere die Obergrenze in der Variable `obergrenze`
- Spiel erzeugt eine Zufallszahl zwischen 1 und Obergrenze
 - Erzeuge eine Zufallszahl zwischen 1 und der angegebenen `obergrenze`
 - Speichere die Zufallszahl in der Variable `zielZahl`
 - Setze die Variable `zähler` für die Rateversuche auf 0

2. Spiel läuft

- Solange die Zahl nicht erraten wurde
 - Benutzer:in zu einem Rateversuch auffordern
 - Gib aus: "Rate die Zahl:"
 - Lies den Rateversuch der Benutzer:in ein
 - Speichere den Rateversuch in der Variable `rateversuch`
 - Erhöhe den `zähler` für die Rateversuche um 1
 - Rateversuch mit Zufallszahl vergleichen und Ergebnis ausgeben
 - Wenn der `rateversuch` kleiner ist als die `zielZahl`:
 - Gib aus: "Die gesuchte Zahl ist größer."
 - Andernfalls, wenn der `rateversuch` größer ist als die `zielZahl`:
 - Gib aus: "Die gesuchte Zahl ist kleiner."
 - Andernfalls (`rateversuch` ist gleich der `zielZahl`):
 - Gib aus: "Richtig geraten! Die gesuchte Zahl war: " + `zielZahl`
 - Beende die Schleife

3. Das Spiel endet

- Gib aus: "Du hast die Zahl in " + `zähler` + " Rateversuchen erraten."

 „Genau, auch super
hilfreich für Codeopolis.“

Genial... Der Vorcode hilft mir, mich zuerst
auf das „Was“ zu konzentrieren!

Gerit – aufgeweckte Student:in im 1. Semester



Von der Idee zur Implementierung



Alles in einem
Durchgang und
auf einmal



Wunschgedanke



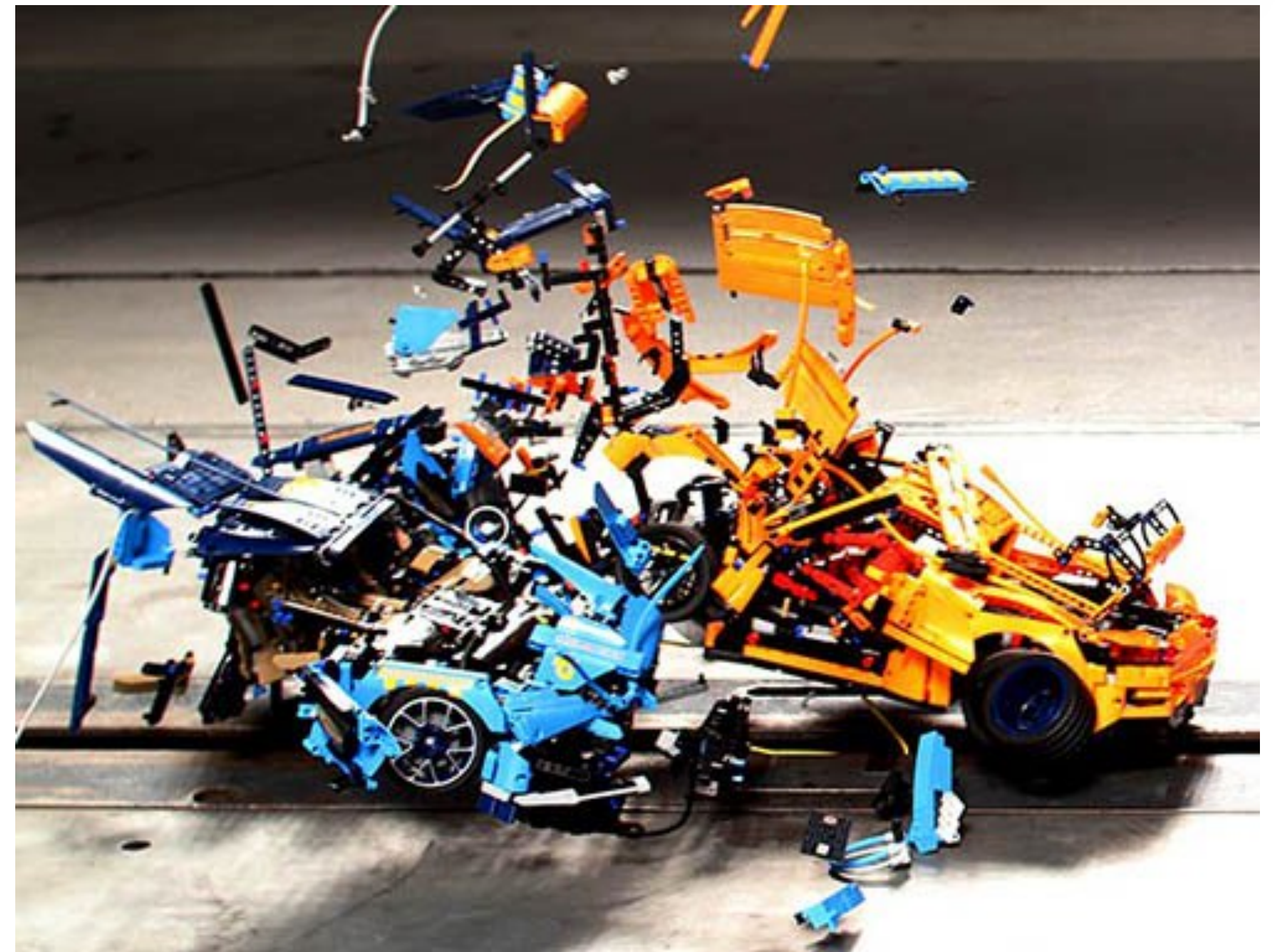
Von der Idee zur Implementierung



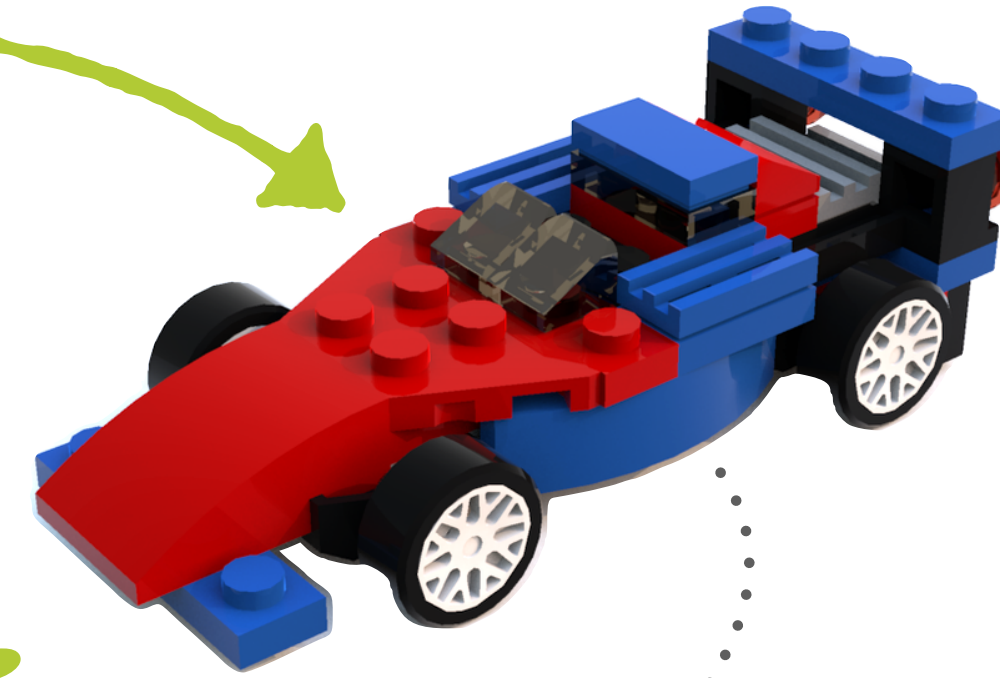
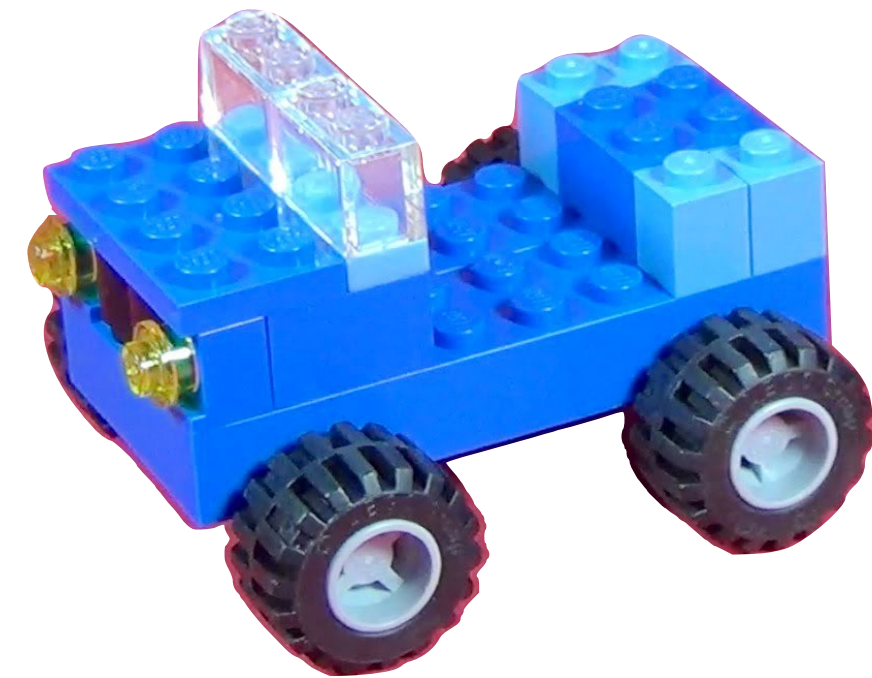
Alles in einem
Durchgang und
auf einmal



Realität



Von der Idee zur Implementierung



iterativ
und



inkrementell



schrittweise und
aufeinander aufbauend



sich wieder-
holendes Vorgehen

„Implementieren Sie sowohl Ihre einzelnen Klassen als auch Ihr Programm in einer sich wiederholenden und aufeinander aufbauenden Weise.“

Maxi – professionelle Entwickler:in, die andere zu Spitzenleistungen führt





„Genau, ein iteratives und
inkrementelles Vorgehen ist auch
super hilfreich für Codeopolis.“



„Mmh... Ich versuche **nicht**, alles in einem
Durchgang und auf einmal zu erledigen!“

Merle – aufmerksame Student:in im 1. Semester



! ? Quiz

👤 Eine Klasse entwickeln

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Wenn Sie eine neue Klasse erstellen,
schreiben Sie typischerweise drei Dinge:
Vorcode, Testcode und echten (Java-)Code.

Richtig!

Bevor ich mich mit dem „Wie“ befasse,
schaue ich mir das „Was“ an.

Richtig!

Der Vorcode sollte genau wiedergeben, wie
Ihr Programm die Lösung in Java
implementiert.

Der Vorcode sollte das „Was“ beschreiben. Die Implementierung in Java, d. h. das „Wie“, erfolgt später.

Ein iterativer Ansatz ist ein sich wiederholender Ansatz. Ein iterativer Entwicklungsprozess wird in sich wiederholenden Zyklen durchgeführt.

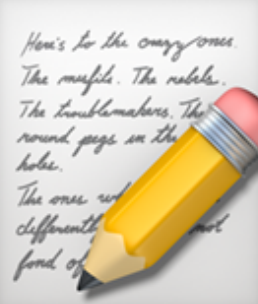
Richtig!

Inkrementell bedeutet schrittweise und aufeinander aufbauend. Ein inkrementeller Entwicklungsprozess fügt also Schritt für Schritt neue Funktionalität hinzu und erweitert die Softwarelösung nach und nach.

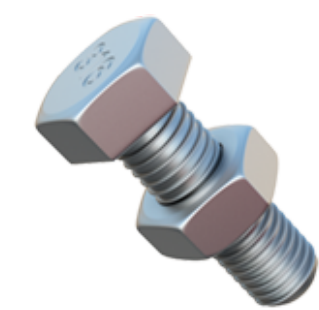
Richtig!

Ein iteratives und inkrementelles Vorgehen ermöglicht eine rasche Rückmeldung, ob man sich „in die richtige Richtung bewegt“.

Richtig!

 Welcher **eine** Gedanke zu
„ Eine Klasse entwickeln“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren



Testcode

Eins nach dem anderen

🔧 Testcode

💡 Schreiben Sie Code zum **Testen** der Methoden bevor Sie die „richtige“ Klasse schreiben.

💡 Genau, Sie schreiben den Testcode, **bevor** es überhaupt etwas zu testen gibt.

💡 Diese **testgetriebene Entwicklung*** erleichtert und beschleunigt Ihre Arbeit.



Ach du liebe Zeit!
Einen Moment lang habe
ich gedacht, Sie würden
den Testcode nicht zuerst
schreiben. Puh! Erschrecken
Sie mich nicht so.

*Der Artikel Test Driven Development von 🧙 Martin Fowler bietet eine gute Übersicht.

„Beginnen Sie damit, sich zu fragen, welche Tests Sie schreiben könnten, um zu zeigen, dass Ihr Code so funktioniert, wie Sie es erwarten.“

Maxi – professionelle Entwickler:in, die andere zu Spitzenleistungen führt





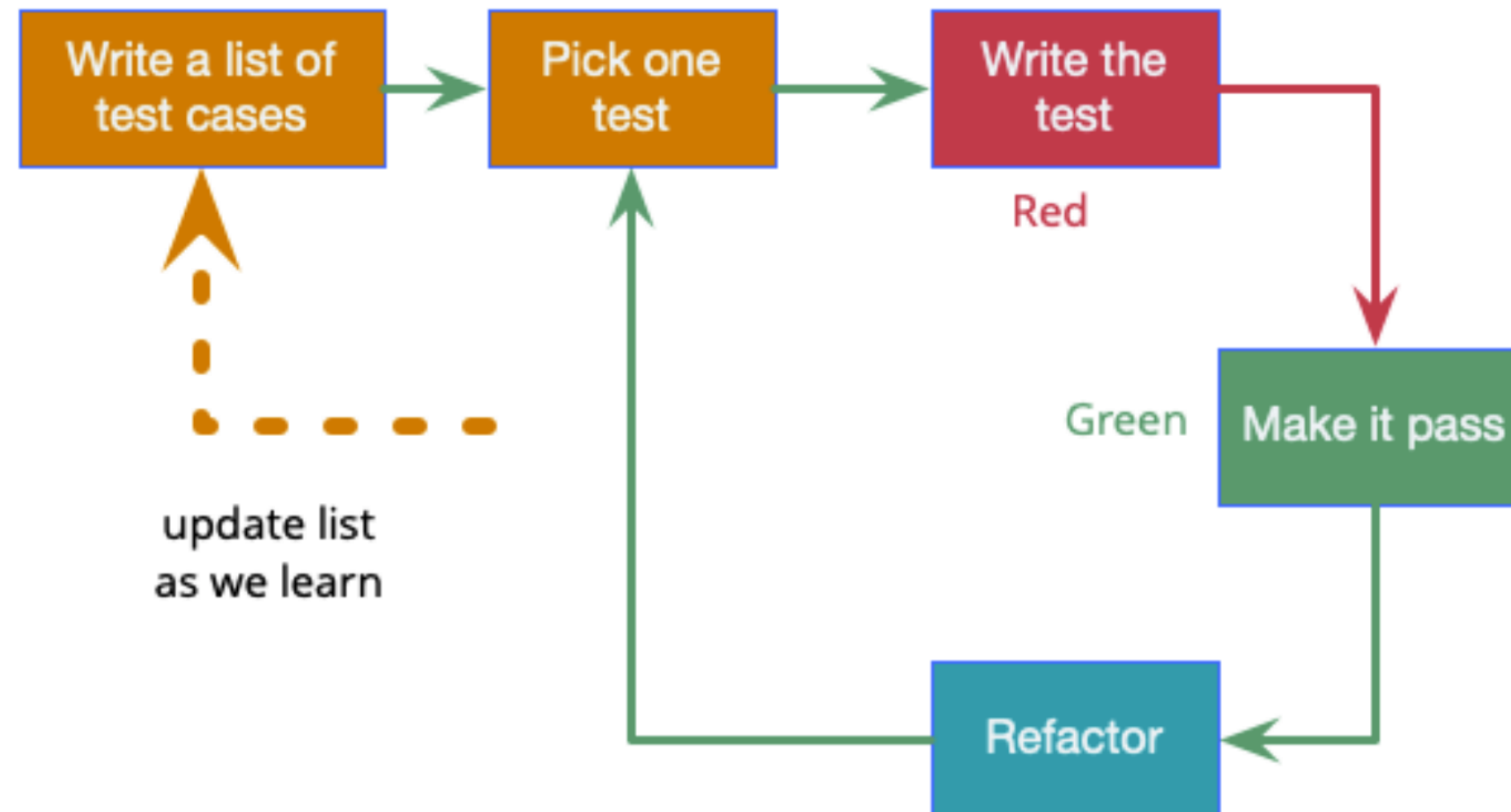
Martin Fowler

◇ TESTING

◇ PROGRAMMING STYLE

Test-Driven Development (TDD) is a technique for building software that guides software development by writing tests. It was developed by Kent Beck in the late 1990's as part of Extreme Programming. In essence we follow three simple steps repeatedly:

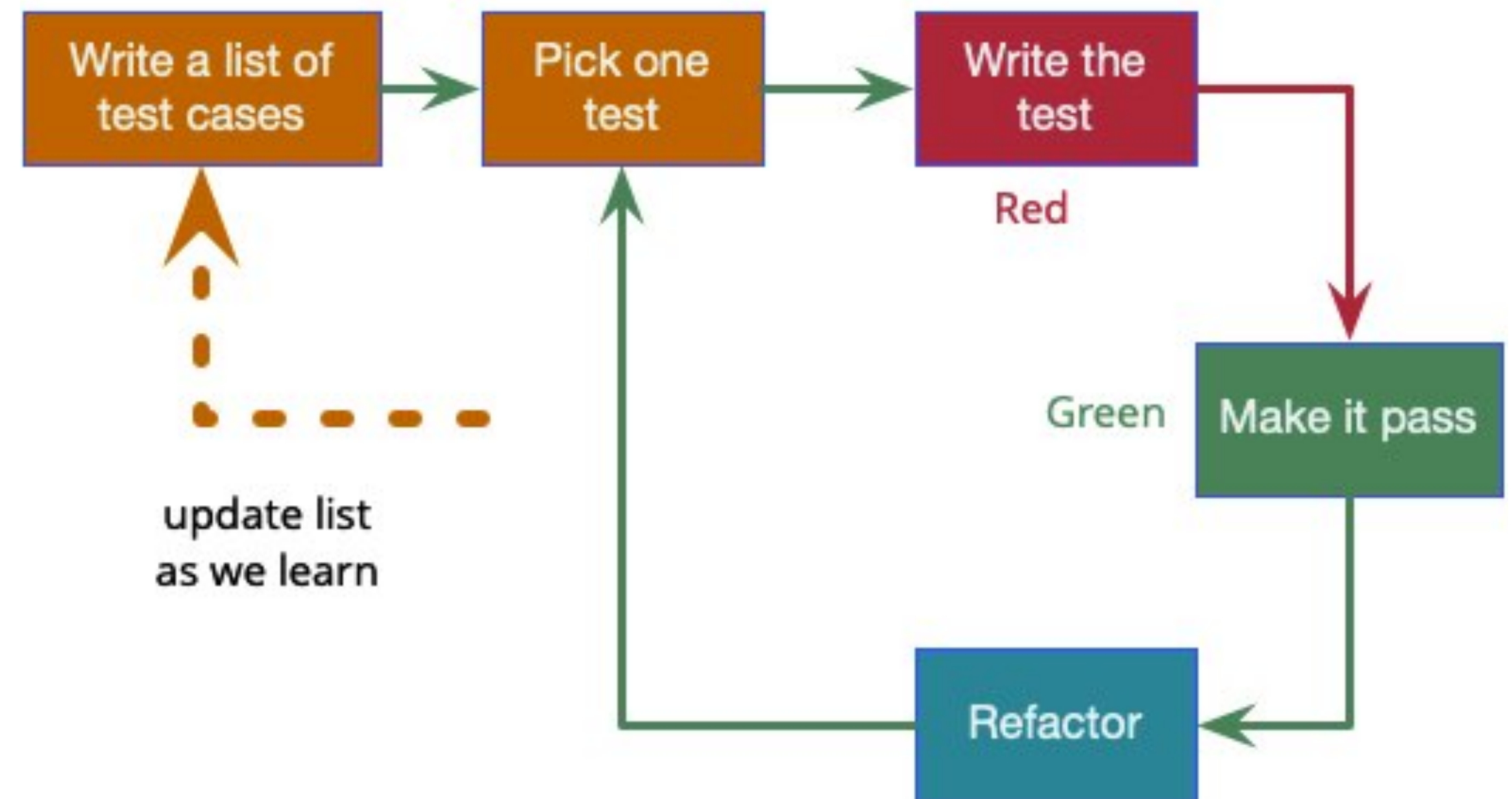
- Write a test for the next bit of functionality you want to add.
- Write the functional code until the test passes.
- Refactor both new and old code to make it well structured.



Testgetriebene Entwicklung

🔩 Testcode

- 🧘 Halten Sie den Code so einfach wie möglich (KISS = Keep it Short and Simple).
- 👋 Bauen Sie nichts ein, was nicht in der Spezifikation steht. Egal wie groß die Versuchung ist.
- 🔄 Wiederholen Sie den Prozess iterativ für jede neue Funktionalität oder Änderung (jedes Inkrement).
- 🛟 Die Tests dienen als ausführbare Spezifikation und als Sicherheitsnetz, um sicherzustellen, dass die vorhandene Funktionalität trotz Änderungen intakt bleibt.
- 💎 Indem Sie sich auf die wesentlichen Anforderungen und deren qualitativ hochwertige Umsetzung konzentrieren, erreichen Sie in der zur Verfügung stehenden Zeit die richtigen Ziele.



 „Genau, und Tests als Sicherheitsnetz sind auch super hilfreich für Codeopolis.“

„Dieses Vorgehen hilft mir, meine Gedanken zu ordnen und zu klären, was mein Code eigentlich tun soll.“

Merle – aufmerksame Student:in im 1. Semester





```
1 import java.util.*;
2
3 public class DoublesDemo {
4     public static void main(String[] args) {
5         double d1 = 0.1;
6         double d2 = 0.2;
7
8         double sum = d1 + d2; // 0.1 + 0.2
9
10        System.out.println("😱 0.1 + 0.2 =");
11        System.out.println(sum);
12    }
13 }
14
```

STDIN

Input for the program (Optional)

Output:

Click on RUN button to see the output

```
1 import java.util.*;
2
3 public class TemperatureConverterTestDrive {
4
5     public static void main(String[] args) {
6         TemperatureConverter converter = new TemperatureConverter();
7
8         System.out.println("Testing Celsius -> Fahrenheit");
9
10        System.out.println("Convert 0°C and 100°C to Fahrenheit (freezing and boiling points)");
11        test(32.0, converter.convertCelsiusToFahrenheit(0.0));
12        test(212.0, converter.convertCelsiusToFahrenheit(100.0));
13
14        System.out.println("Convert -40°C to Fahrenheit (special case where Celsius equals Fahrenheit)");
15        test(-40.0, converter.convertCelsiusToFahrenheit(-40.0));
16
17        System.out.println("Convert typical Celsius values to Fahrenheit");
18        test(69.8, converter.convertCelsiusToFahrenheit(21.0));
19        test(23.0, converter.convertCelsiusToFahrenheit(-5.0));
20
21        System.out.println("Convert decimal Celsius values to Fahrenheit");
22        test(116.798, converter.convertCelsiusToFahrenheit(47.11));
23        test(17.33, converter.convertCelsiusToFahrenheit(-8.15));
24
25        System.out.println();
26        System.out.println("Testing Fahrenheit -> Celsius");
27
28        System.out.println("Convert 32°F and 212°F to Celsius (freezing and boiling points)");
29        test(0.0, converter.convertFahrenheitToCelsius(32.0));
30        test(100.0, converter.convertFahrenheitToCelsius(212.0));
31
32        System.out.println("Convert -40°F to Celsius (special case where Fahrenheit equals Celsius)");
33        test(-40.0, converter.convertFahrenheitToCelsius(-40.0));
34
35        System.out.println("Convert typical Fahrenheit values to Celsius");
36        test(21.0, converter.convertFahrenheitToCelsius(69.8));
37        test(-5.0, converter.convertFahrenheitToCelsius(23.0));
38
39        System.out.println("Convert decimal Fahrenheit values to Celsius");
40        test(47.11, converter.convertFahrenheitToCelsius(116.798));
41        test(-8.15, converter.convertFahrenheitToCelsius(17.33));
42    }
43
44    public static void test(double expected, double actual) {
45        // Kurs erklärt für Java-Anfänger:innen: Was muss ich beachten,
46        // wenn ich die Ergebnisse zweier double-Berechnungen vergleiche?
47        // https://www.perplexity.ai/search/kurs-erklart-fur-java-anfanger-MXa8bKyPQGu0yZ.rTe2cQA
48        if (Math.abs(expected - actual) < 0.000001d) {
49            System.out.println(
50                "✅ Expected: " + expected + " - Actual: " + actual);
51        } else {
52            System.out.println(
53                "❌ Expected: " + expected + " - Actual: " + actual);
54        }
55    }
56
57 }
58
```

STDIN

Input for the program (Optional)

Output:

Testing Celsius -> Fahrenheit
Convert 0°C and 100°C to Fahrenheit (freezing and boiling points)
❌ Expected: 32.0 – Actual: 0.0
❌ Expected: 212.0 – Actual: 0.0
Convert -40°C to Fahrenheit (special case where Celsius equals Fahrenheit)
❌ Expected: -40.0 – Actual: 0.0
Convert typical Celsius values to Fahrenheit
❌ Expected: 69.8 – Actual: 0.0
❌ Expected: 23.0 – Actual: 0.0
Convert decimal Celsius values to Fahrenheit
❌ Expected: 116.798 – Actual: 0.0
❌ Expected: 17.33 – Actual: 0.0

Testing Fahrenheit -> Celsius
Convert 32°F and 212°F to Celsius (freezing and boiling points)
❌ Expected: 0.0 – Actual: 1.0
❌ Expected: 100.0 – Actual: 1.0
Convert -40°F to Celsius (special case where Fahrenheit equals Celsius)
❌ Expected: -40.0 – Actual: 1.0
Convert typical Fahrenheit values to Celsius
❌ Expected: 21.0 – Actual: 1.0
❌ Expected: -5.0 – Actual: 1.0
Convert decimal Fahrenheit values to Celsius
❌ Expected: 47.11 – Actual: 1.0
❌ Expected: -8.15 – Actual: 1.0



„Genau, so ein schrittweises
Vorgehen ist auch für
Codeopolis super hilfreich.“



„Moment, hieß es nicht, wir sollen einen
Test nach dem anderen abhaken?“

Karli – kritische Student:in im 1. Semester




```
1 import java.util.*;
2
3 public class TemperatureConverterTestDrive {
4
5     public static void main(String[] args) {
6         TemperatureConverter converter = new TemperatureConverter();
7
8         System.out.println("Testing Celsius -> Fahrenheit");
9
10        System.out.println("Convert 0°C and 100°C to Fahrenheit (freezing and boiling points)");
11        test(32.0, converter.convertCelsiusToFahrenheit(0.0));
12        test(212.0, converter.convertCelsiusToFahrenheit(100.0));
13
14        System.out.println("Convert -40°C to Fahrenheit (special case where Celsius equals Fahrenheit)");
15        test(-40.0, converter.convertCelsiusToFahrenheit(-40.0));
16
17        System.out.println("Convert typical Celsius values to Fahrenheit");
18        test(69.8, converter.convertCelsiusToFahrenheit(21.0));
19        test(23.0, converter.convertCelsiusToFahrenheit(-5.0));
20
21        System.out.println("Convert decimal Celsius values to Fahrenheit");
22        test(116.798, converter.convertCelsiusToFahrenheit(47.11));
23        test(17.33, converter.convertCelsiusToFahrenheit(-8.15));
24    }
25
26    public static void test(double expected, double actual) {
27        // Kurs erklärt für Java-Anfänger:innen: Was muss ich beachten,
28        // wenn ich die Ergebnisse zweier double-Berechnungen vergleiche?
29        // https://www.perplexity.ai/search/kurs-erklart-fur-java-anfanger-MXa8bKyPQGu0yZ.rTe2cQA
30        if (Math.abs(expected - actual) < 0.000001d) {
31            System.out.println(
32                "✅ Expected: " + expected + " - Actual: " + actual);
33        } else {
34            System.out.println(
35                "❌ Expected: " + expected + " - Actual: " + actual);
36        }
37    }
38
39 }
40
```

STDIN

Input for the program (Optional)

Output:

```
Testing Celsius -> Fahrenheit
Convert 0°C and 100°C to Fahrenheit (freezing and boiling points)
❌ Expected: 32.0 - Actual: 0.0
❌ Expected: 212.0 - Actual: 0.0
Convert -40°C to Fahrenheit (special case where Celsius equals Fahrenheit)
❌ Expected: -40.0 - Actual: 0.0
Convert typical Celsius values to Fahrenheit
❌ Expected: 69.8 - Actual: 0.0
❌ Expected: 23.0 - Actual: 0.0
Convert decimal Celsius values to Fahrenheit
❌ Expected: 116.798 - Actual: 0.0
❌ Expected: 17.33 - Actual: 0.0
```



„Wir fokussieren auf
eine Richtung: Celsius zu
Fahrenheit“



„Genau, je kleiner die Schritte, desto höher der Fokus und desto geringer das Risiko.“



„Ha! Es geht auch in noch kleineren Schritten!“

Gerit – aufgeweckte Student:in im 1. Semester



```
1 import java.util.*;
2
3 public class TemperatureConverterTestDrive {
4
5     public static void main(String[] args) {
6         TemperatureConverter converter = new TemperatureConverter();
7
8         System.out.println("Testing Celsius -> Fahrenheit");
9
10        System.out.println("Convert 0°C and 100°C to Fahrenheit (freezing and boiling points)");
11        test(32.0, converter.convertCelsiusToFahrenheit(0.0));
12        test(212.0, converter.convertCelsiusToFahrenheit(100.0));
13    }
14
15    public static void test(double expected, double actual) {
16        // Kurs erklärt für Java-Anfänger:innen: Was muss ich beachten,
17        // wenn ich die Ergebnisse zweier double-Berechnungen vergleiche?
18        // https://www.perplexity.ai/search/kurs-erklart-fur-java-anfanger-MXa8bKyPQGu0yZ.rTe2cQA
19        if (Math.abs(expected - actual) < 0.000001d) {
20            System.out.println(
21                "✅ Expected: " + expected + " - Actual: " + actual);
22        } else {
23            System.out.println(
24                "❌ Expected: " + expected + " - Actual: " + actual);
25        }
26    }
27
28 }
29
```

STDIN

Input for the program (Optional)

Output:

```
Testing Celsius -> Fahrenheit
Convert 0°C and 100°C to Fahrenheit (freezing and boiling points)
❌ Expected: 32.0 - Actual: 0.0
❌ Expected: 212.0 - Actual: 0.0
```



„Wir fokussieren auf
ein Thema.“


```
1 import java.util.*;
2
3 public class TemperatureConverterTestDrive {
4
5     public static void main(String[] args) {
6         TemperatureConverter converter = new TemperatureConverter();
7
8         System.out.println("Testing Celsius -> Fahrenheit");
9
10        System.out.println("Convert 0°C and 100°C to Fahrenheit (freezing and boiling points)");
11        test(32.0, converter.convertCelsiusToFahrenheit(0.0));
12    }
13
14    public static void test(double expected, double actual) {
15        // Kurs erklärt für Java-Anfänger:innen: Was muss ich beachten,
16        // wenn ich die Ergebnisse zweier double-Berechnungen vergleiche?
17        // https://www.perplexity.ai/search/kurs-erklart-fur-java-anfanger-MXa8bKyPQGy0yZ.rTe2cQA
18        if (Math.abs(expected - actual) < 0.000001d) {
19            System.out.println(
20                "✅ Expected: " + expected + " - Actual: " + actual);
21        } else {
22            System.out.println(
23                "❌ Expected: " + expected + " - Actual: " + actual);
24        }
25    }
26
27 }
28
```

STDIN

Input for the program (Optional)

Output:

```
Testing Celsius -> Fahrenheit
Convert 0°C and 100°C to Fahrenheit (freezing and boiling points)
❌ Expected: 32.0 - Actual: 0.0
```



„Wir fokussieren auf einen einzigen Aspekt.“

! ? Quiz

🔧 Testcode

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Testcode ist eine Klasse oder sind Methoden,
die den Hauptcode testen, um sicherzustellen,
dass er das Richtige richtig tut.

Richtig!

Sie können Ihren Testcode von Ihrem Vorcode ableiten.

Richtig!

Testgetriebene Entwicklung (TDD) ist aufwändig und daher nur für Expert:innen geeignet.

TDD kann Ihre Arbeit einfacher und schneller machen.

TDD beginnt mit der Frage, woran man erkennen kann, dass das Programm das „Was“ richtig macht. Dazu wird eine erste Liste von Testfällen erstellt.

Richtig!

Nach Erstellung dieser Liste ist der TDD-Zyklus genau einmal zu wiederholen, wobei alle Tests zu implementieren sind.

Sie wenden den TDD-Zyklus jeweils nur auf einen Testfall an.

Sie schreiben einen Test für die nächste Funktionalität, die Sie hinzufügen möchten. („Red“)

Richtig!

Nach dem Test schreiben Sie den produktiven Code (die Anwendungslogik), bis der Test erfolgreich ist. („Green“)

Richtig!

Im letzten Schritt wird ausschließlich der neue Code überarbeitet, um eine gute Struktur zu gewährleisten. („Refactor“)

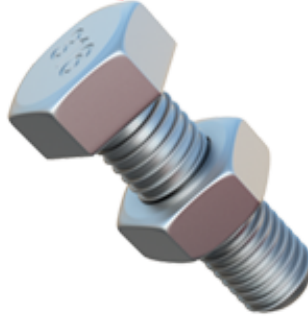
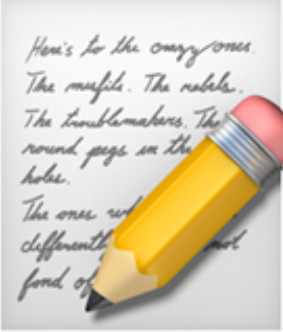
Sowohl der neue als auch der alte Code werden refaktorisiert.

TDD hilft, Overengineering oder das „vorschnelle“ Schreiben von Code, der noch nicht durch einen Test motiviert ist, zu vermeiden.

Richtig!

Ähnlich einem Sicherheitsnetz, das Artist:innen vor einem Absturz schützt, minimieren Tests das Risiko von Fehlern und unbeabsichtigten Änderungen durch die Entwickler:innen.

Richtig!

 Welcher **eine** Gedanke zu
„ Testcode“ war für Sie am
wichtigsten?  Schreiben Sie
ihn auf!

 Durchatmen und  Sortieren

Hauptcode

```
1 public class TemperatureConverter {
2
3     public double convertCelsiusToFahrenheit(double celsius) {
4         return (celsius * 9 / 5) + 32;
5     }
6
7     public double convertFahrenheitToCelsius(double fahrenheit) {
8         return (fahrenheit - 32) * 5 / 9;
9     }
10
11 }
12
```

STDIN

Input for the program (Optional)

Output:

Testing Celsius -> Fahrenheit
Convert 0°C and 100°C to Fahrenheit (freezing and boiling points)
✔ Expected: 32.0 – Actual: 32.0
✔ Expected: 212.0 – Actual: 212.0
Convert -40°C to Fahrenheit (special case where Celsius equals Fahrenheit)
✔ Expected: -40.0 – Actual: -40.0
Convert typical Celsius values to Fahrenheit
✔ Expected: 69.8 – Actual: 69.8
✔ Expected: 23.0 – Actual: 23.0
Convert decimal Celsius values to Fahrenheit
✔ Expected: 116.798 – Actual: 116.798
✔ Expected: 17.33 – Actual: 17.33

Testing Fahrenheit -> Celsius
Convert 32°F and 212°F to Celsius (freezing and boiling points)
✔ Expected: 0.0 – Actual: 0.0
✔ Expected: 100.0 – Actual: 100.0
Convert -40°F to Celsius (special case where Fahrenheit equals Celsius)
✔ Expected: -40.0 – Actual: -40.0
Convert typical Fahrenheit values to Celsius
✔ Expected: 21.0 – Actual: 21.0
✔ Expected: -5.0 – Actual: -5.0
Convert decimal Fahrenheit values to Celsius
✔ Expected: 47.11 – Actual: 47.11
✔ Expected: -8.15 – Actual: -8.15



„Genau, das sehen wir
am Donnerstag in Teil 2.“

„Hey! Und den Hauptcode entwickeln wir
auch iterativ und inkrementell. Also
mittels TDD.“

Gerit – aufgeweckte Student:in im 1. Semester



! ? Quiz

🔧 Hauptcode

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Der Hauptcode folgt als Produktionscode auf den Testcode und stellt somit das „Was“ dar.

Der Hauptcode steht dabei für das „Wie“.

Der Hauptcode ist die eigentliche Implementierung der Anwendungslogik mit entsprechendem Java-Code.

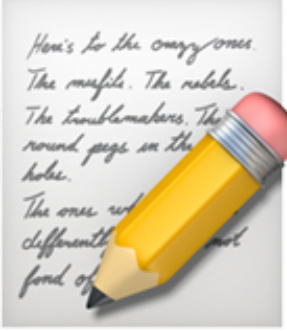
Richtig!

Auch der Hauptcode wird iterativ und
inkrementell erstellt.

Richtig!

TDD bietet eine geeignete Möglichkeit, den Hauptcode schrittweise und aufeinander aufbauend über mehrere Zyklen zu schreiben.

Richtig!

 Welcher **eine** Gedanke zu
„ Hauptcode“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Ein Programm schreiben

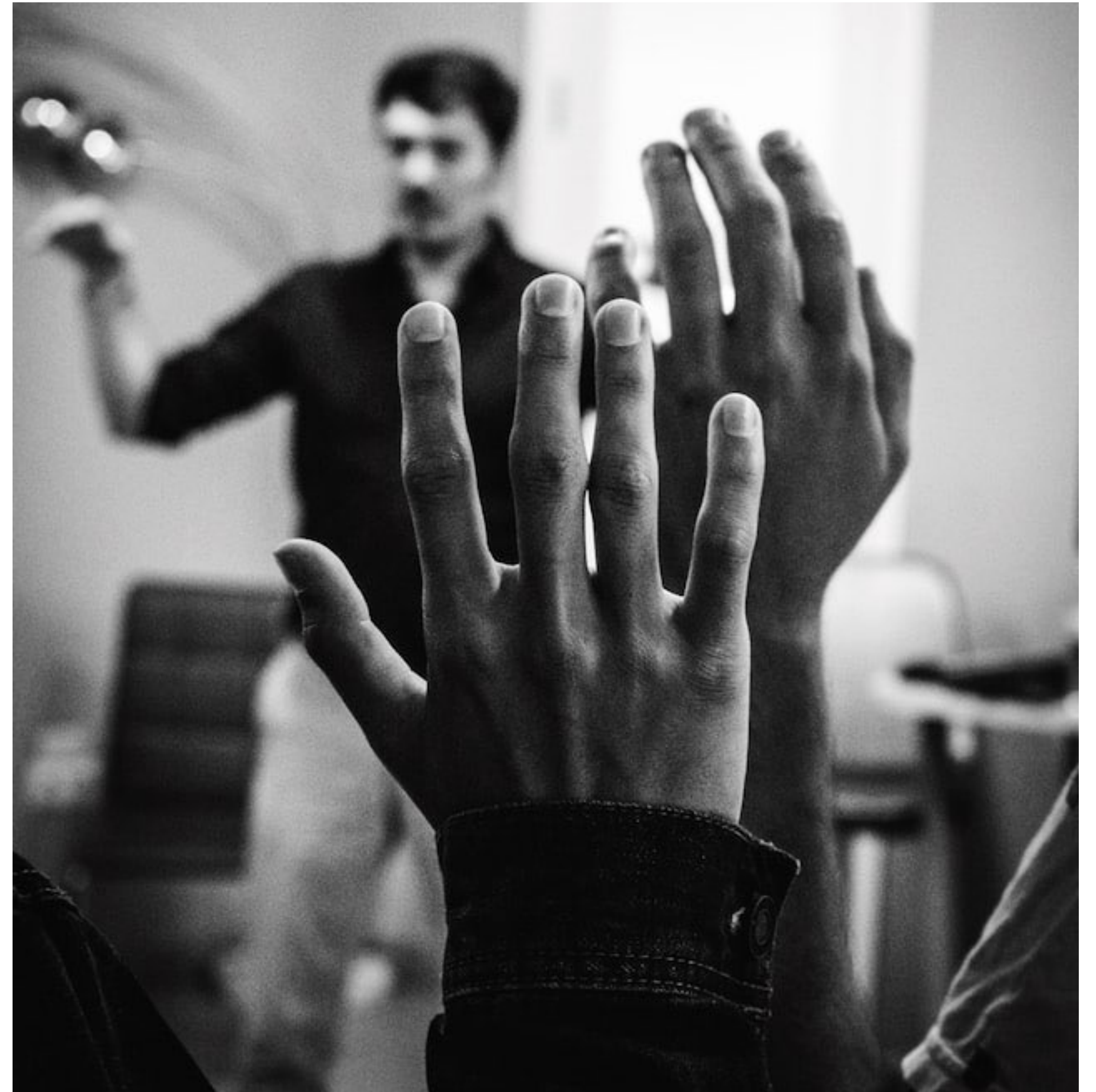
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Ein Programm schreiben?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Ein Programm schreiben



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

